P2216R1
std::format improvements

Published Proposal, 2020-11-25

This version:
http://fmt.dev/papers/p2216r1.html

Author:
Victor Zverovich

Audience:
LEWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Table of Contents

1 Introduction

2 Revision history

3 LEWG polls (R0)

4 Compile-time checks

5 Binary size

6 Impact on existing code

7 Wording

8 Acknowledgements

References

Informative References

"Safety doesn’t happen by accident."
— unknown

§ 1. Introduction

This paper proposes the following improvements to the C++20 formatting facility:

- Improving safety via compile-time format string checks
- Reducing binary code size of `format_to`

§ 2. Revision history

Changes since R0:

- Removed "Passing an argument `fmt` that is not a format string for parameter pack `args` is ill-formed with no diagnostic required." per LEWG feedback.
- Changed the wording to use C++20 facilities and an exposition-only *`basic_format_string`* type for guaranteed diagnostic and no reliance on compiler extensions per LEWG feedback.
- Added an implementation sketch to § 4 Compile-time checks.
- Clarified why code bloat cannot be addressed just as a quality of implementation issue in § 5 Binary size.
- Added an example illustrating one of the cases where code bloat occurs to § 5 Binary size.

§ 3. LEWG polls (R0)

We should promise more committee time to pursuing the compile time checking aspects of P2216R0, knowing that our time is scarce and this will leave less time for other work.

SF	F	N	A	SA
6	6	3	0	0

Consensus to pursue

We should promise more committee time to pursuing the code bloat aspects of P2216R0, knowing that our time is scarce and this will leave less time for other work.

SF	F	N	A	SA
3	8	6	0	0

Consensus to pursue

We are comfortable having `std::format` compile time check failures cause the program to be ill-formed, no diagnostic required (IFNDR).

SF	F	N	A	SA
0	1	2	4	8

LEWG is not comfortable with IFNDR

LEWG would prefer `std::format` compile time check failures to cause the program to be ill-formed (diagnostic required).

SF	F	N	A	SA
5	7	1	0	0

LEWG prefers ill-formed

We are comfortable having `std::format` compile time checks rely on compiler extensions to be implementable.

SF	F	N	A	SA
3	3	4	4	0

LEWG is somewhat is uncomfortable with relying on compiler extensions for this facility

§ 4. Compile-time checks

Consider the following example:

```
std::string s = std::format("{:d}", "I am not a number");
```

In C++20 ([N4861]) it throws `format_error` because `d` is not a valid format specifier for a null-terminated character string.

We propose making it ill-formed resulting in a compile-time rather than a runtime error. This will significantly improve safety of the formatting API and bring it on par with other languages such as D ([D-FORMAT]) and Rust ([RUST-FMT]).

This proposal has been successfully implemented in the open-source {fmt} library ([FMT]) using only C++20 facilities and tested on Clang 11 and GCC 10. It will become the default in the next major release of the library. The implementation is very simple and straightforward because format string parsing in C++20 has been designed with such checks in mind ([P0645]) and is already `constexpr`.

There are two options:

1. Provide compile-time checks for all format strings known at compile time.
2. Limit checks to string literals only.

Here is a sketch of the implementation:

```
#ifdef OPTION_1 // exposition only

// Option 1:
template<class charT, class... Args> struct basic_format_string {
  basic_string_view<charT> str;

  template<class T, enable_if_t<
    is_convertible_v<const T&, basic_string_view<charT>>, int> = 0>
  consteval basic_format_string(const T& s) : str(s) {
    // Report a compile-time error if s is not a format string for Args.
  }
};

#else

// Option 2:
template<class charT, class... Args> struct basic_format_string {
  basic_string_view<charT> str;

  template<size_t N>
  consteval basic_format_string(const charT (&s)[N]) : str(s) {
    // Report a compile-time error if s is not a format string for Args.
  }

  template<class T, enable_if_t<
    is_convertible_v<const T&, basic_string_view<charT>>, int> = 0>
  basic_format_string(const T& s) : str(s) {}
};

#endif

// Same for Option 1 & Option 2:
template<class... Args>
  using format_string =
    basic_format_string<char, type_identity_t<Args>...>;

template <class... Args>
string format(format_string<Args...> fmt, const Args&... args) {
  return vformat(fmt.str, make_format_args(args...));
}
```

Compiling our example produces the following diagnostic on Clang:

```
<source>:36:26: error: call to consteval function 'basic_format_string<char
  std::string s = format("{:d}", "I am not a number");
                         ^
/opt/compiler-explorer/libs/fmt/trunk/include/fmt/format.h:1422:13: note: r
    handler.on_error("invalid type specifier");
            ^
...
```

Comparison of different options:

Code	C++20	Option 1	Option 2
`auto s = format("{:d}", 42);`	OK	OK	OK
`auto s = format("{:s}", 42);`	throws	ill-formed	ill-formed
`constexpr const char fmt[] = "{:d}";` `auto s = format(fmt, 42);`	OK	OK	OK
`const char fmt[] = "{:d}";` `auto s = format(fmt, 42);`	OK	ill-formed	ill-formed
`constexpr const char* fmt = "{:s}";` `auto s = format(fmt, 42);`	throws	ill-formed	throws
`const char* fmt = "{:d}";` `auto s = format(fmt, 42);`	OK	ill-formed	OK

Option 1 is safer but has the same limitation as Rust's `format!` of only accepting format strings known at compile time. However, it is still possible to pass a runtime string via `vformat`:

```
const char* fmt = "{:d}";
auto s = vformat(fmt, make_format_args(42));
```

Additionally we can provide a convenience wrapper for passing runtime strings:

```
const char* fmt = "{:d}";
auto s = format(runtime_format(fmt), 42);
```

Note that in the vast majority of cases format strings are literals. For example, analyzing a sample of 100 `printf` calls from [CODESEARCH] showed that 98 of them are string literals and 2 are string literals wrapped in the `_` gettext macro:

```
printf (_("call to tc_aout_fix_to_chars \n"));
```

In this case translation and runtime format markers can be combined without any impact on usability.

We propose making `basic_format_string` exposition-only because it is an implementation detail and in the future the same functionality can be implemented using [P1221] (see e.g. https://godbolt.org/z/hcnxfY) or [P1045].

From the extensive usage experience in the {fmt} library ([FMT]) that provides compile-time checks as an opt-in we've found that users expect errors in literal format strings to be diagnosed at compile time by default. One of the reasons is that such diagnostic is commonly done in `printf`, for example:

```
printf("%d", "I am not a number");
```

gives a warning both in GCC and clang:

```
warning: format specifies type 'int' but the argument has type 'const char
```

so users expect the same or better level of diagnostics from a similar C++ facility.

§ 5. Binary size

The `vformat_to` functions take format arguments parameterized on the output iterator via the formatting context:

```
template<class Out, class charT>
    using format_args_t = basic_format_args<basic_format_context<Out, charT

template<class Out>
  Out vformat_to(Out out, string_view fmt,
                 format_args_t<type_identity_t<Out>, char> args);
```

Unfortunately it may result in significant code bloat because formatting code will have to be instantiated for every iterator type used with `format_to` or `vformat_to`, for example:

```
std::vector<char> v;
std::format_to(std::back_inserter(v), "{}", 42);
// Formatting functions are instantiated for std::back_insert_iterator<std:

std::string s;
std::format_to(std::back_inserter(s), "{}", 42);
// Formatting functions are instantiated for std::back_insert_iterator<std:
```

This happens even for argument types that are not formatted, clearly violating "you don't pay for what you don't use" principle. Also this is unnecessary because the iterator type can be erased via the internal buffer as it is done in `format` and `vformat` without affecting performance for the common case of containers with contiguous storage. Therefore we propose using `format_args` and `wformat_args` instead of `format_args_t` in these overloads:

```
template<class Out>
  Out vformat_to(Out out, string_view fmt, format_args args);
```

`formatter` specializations will continue to support output iterators so this only affects type-erased API and not the one with compiled format strings that will be proposed separately. The latter will not be affected by the code bloat issue because instantiations will be limited only to used argument types.

In addition to reducing the code bloat this will simplify the API.

The code bloat problem cannot be solved just as a quality of implementation issue because the iterator type is observable through the `formatter` API.

This proposal has been successfully implemented in the {fmt} library ([FMT]).

§ 6. Impact on existing code

Making invalid format strings ill-formed and modifying the problematic `vformat_to` overloads are breaking changes although at the time of writing none of the standard libraries implements the C++20 formatting facility and therefore there is no code using it.

§ 7. Wording

All wording is relative to the C++ working draft [N4861].

Update the value of the feature-testing macro `__cpp_lib_format` to the date of adoption in [version.syn]:

Change in [format.syn]:

```
namespace std {
  // 20.20.3, error reporting
  template<class charT, class... Args> struct basic-format-string {  // exp
    basic_string_view<charT> str;

    template<class T> consteval basic-format-string(const T& s);
  };

  template<class... Args>
    using format-string =
      basic-format-string<char, type_identity_t<Args>...>;     // expositio
  template<class... Args>
    using wformat-string =
      basic-format-string<wchar_t, type_identity_t<Args>...>;  // expositio

  // 20.20.4, formatting functions
  template<class... Args>
    string format(~~string_view~~format-string<Args...> fmt, const Args&... arg
  template<class... Args>
    wstring format(~~wstring_view~~wformat-string<Args...> fmt, const Args&...
  template<class... Args>
    string format(const locale& loc, ~~string_view~~format-string<Args...> fmt,
                  const Args&... args);
  template<class... Args>
    wstring format(const locale& loc, ~~wstring_view~~wformat-string<Args...> f
                   const Args&... args);

  ...

  template<class Out, class... Args>
    Out format_to(Out out, ~~string_view~~format-string<Args...> fmt, const Arg
  template<class Out, class... Args>
    Out format_to(Out out, ~~wstring_view~~wformat-string<Args...> fmt, const A
  template<class Out, class... Args>
    Out format_to(Out out, const locale& loc, ~~string_view~~format-string<Args
                  const Args&... args);
  template<class Out, class... Args>
    Out format_to(Out out, const locale& loc, ~~wstring_view~~wformat-string<Ar
                  const Args&... args);

  template<class Out>
    Out vformat_to(Out out, string_view fmt,
                   ~~format_args_t<type_identity_t<Out>, char>~~format_args arg
```

```
  template<class Out>
    Out vformat_to(Out out, wstring_view fmt,
                   format_args_t<type_identity_t<Out>, wchar_t>wformat_args
  template<class Out>
    Out vformat_to(Out out, const locale& loc, string_view fmt,
                   format_args_t<type_identity_t<Out>, char>format_args arg
  template<class Out>
    Out vformat_to(Out out, const locale& loc, wstring_view fmt,
                   format_args_t<type_identity_t<Out>, wchar_t>wformat_args

  ...

  template<class Out, class... Args>
    format_to_n_result<Out> format_to_n(Out out, iter_difference_t<Out> n,
                                        string_viewformat-string<Args...> f
                                        const Args&... args);
  template<class Out, class... Args>
    format_to_n_result<Out> format_to_n(Out out, iter_difference_t<Out> n,
                                        wstring_viewwformat-string<Args...>
                                        const Args&... args);
  template<class Out, class... Args>
    format_to_n_result<Out> format_to_n(Out out, iter_difference_t<Out> n,
                                        const locale& loc,
                                        string_viewformat-string<Args...> f
                                        const Args&... args);
  template<class Out, class... Args>
    format_to_n_result<Out> format_to_n(Out out, iter_difference_t<Out> n,
                                        const locale& loc,
                                        wstring_viewwformat-string<Args...>
                                        const Args&... args);

  template<class... Args>
    size_t formatted_size(string_viewformat-string<Args...> fmt, const Args
  template<class... Args>
    size_t formatted_size(wstring_viewwformat-string<Args...> fmt, const Ar
  template<class... Args>
    size_t formatted_size(const locale& loc, string_viewformat-string<Args.
                          const Args&... args);
  template<class... Args>
    size_t formatted_size(const locale& loc, wstring_viewwformat-string<Arg
                          const Args&... args);

  ...
```

```
  // 20.20.6.3, class template basic_format_args
  ...
  template<class Out, class charT>
    using format_args_t = basic_format_args<basic_format_context<Out, charT
```

Change in [format.string.general]:

If all *arg-id*s in a format string are omitted (including those in the *format-spec*, as interpreted by the corresponding `formatter` specialization), argument indices 0, 1, 2, ... will automatically be used in that order. If some *arg-id*s are omitted and some are present, the string is not a format string. [Note: A format string cannot contain a mixture of automatic and manual indexing. — end note] [*Example*:

```
string s0 = format("{} to {}", "a", "b");   // OK, automatic indexing
string s1 = format("{1} to {0}", "a", "b"); // OK, manual indexing
string s2 = format("{0} to {}", "a", "b");  // not a format string (mixing
                                            // throws format_errorill-forme
string s3 = format("{} to {1}", "a", "b");  // not a format string (mixing
                                            // throws format_errorill-forme
```

— *end example*]

Change in [format.err.report]:

... Failure to allocate storage is reported by throwing an exception as described in 16.5.5.13.

```
template<class charT, class... Args> struct basic-format-string {  // expos
  basic_string_view<charT> str;

  template<class T> consteval basic-format-string(const T& s);
};
```

```
template<class T> basic-format-string(const T& s);
```

Constraints: `is_convertible_v<const T&, basic_string_view<charT>>` is `true`.

Mandates: `s` is a format string for `Args`.

Effects: initializes `str` with `s`.

Change in [format.functions]:

```
template<class... Args>
  string format(~~string_view~~format-string<Args...> fmt, const Args&... args)
```

Effects: Equivalent to:

```
return vformat(fmt.str, make_format_args(args...));
```

```
template<class... Args>
  wstring format(~~wstring_view~~wformat-string<Args...> fmt, const Args&... ar
```

Effects: Equivalent to:

```
return vformat(fmt.str, make_wformat_args(args...));
```

```
template<class... Args>
  string format(const locale& loc, ~~string_view~~format-string<Args...> fmt,
                const Args&... args);
```

Effects: Equivalent to:

```
return vformat(loc, fmt.str, make_format_args(args...));
```

```
template<class... Args>
  wstring format(const locale& loc, ~~wstring_view~~wformat-string<Args...> fmt
                 const Args&... args);
```

Effects: Equivalent to:

```
return vformat(loc, fmt.str, make_wformat_args(args...));
```

...

```
template<class Out, class... Args>
  Out format_to(Out out, ~~string_view~~format-string<Args...> fmt, const Args&
```

Effects: Equivalent to:

```
return vformat_to(out, fmt.str, make_format_args(args...));
```

```
template<class Out, class... Args>
  Out format_to(Out out, ~~wstring_view~~wformat-string<Args...> fmt, const Arg
```

Effects: Equivalent to:

```
~~using context = basic_format_context<Out, decltype(fmt)::value_type>;~~
~~return vformat_to(out, fmt.str, make_format_args<context>(args...));~~
return vformat_to(out, fmt, make_wformat_args(args...));
```

```
template<class Out, class... Args>
  Out format_to(Out out, const locale& loc, ~~string_view~~format-string<Args..
                const Args&... args);
```

Effects: Equivalent to:

```
return vformat_to(out, loc, fmt.str, make_format_args(args...));
```

```
template<class Out, class... Args>
  Out format_to(Out out, const locale& loc, ~~wstring_view~~wformat-string<Args
                const Args&... args);
```

Effects: Equivalent to:

```
~~using context = basic_format_context<Out, decltype(fmt)::value_type>;~~
~~return vformat_to(out, loc, fmt, make_format_args<context>(args...));~~
return vformat_to(out, loc, fmt, make_wformat_args(args...));
```

```
template<class Out>
  Out vformat_to(Out out, string_view fmt,
                 format_args_t<type_identity_t<Out>, char>format_args args)
template<class Out>
  Out vformat_to(Out out, wstring_view fmt,
                 format_args_t<type_identity_t<Out>, wchar_t>wformat_args a
template<class Out>
  Out vformat_to(Out out, const locale& loc, string_view fmt,
                 format_args_t<type_identity_t<Out>, char>format_args args)
template<class Out>
  Out vformat_to(Out out, const locale& loc, wstring_view fmt,
                 format_args_t<type_identity_t<Out>, wchar_t>wformat_args a
```

...

```
template<class Out, class... Args>
  format_to_n_result<Out> format_to_n(Out out, iter_difference_t<Out> n,
                                      string_viewformat-string<Args...> fmt
                                      const Args&... args);
template<class Out, class... Args>
  format_to_n_result<Out> format_to_n(Out out, iter_difference_t<Out> n,
                                      wstring_viewwformat-string<Args...> f
                                      const Args&... args);
template<class Out, class... Args>
  format_to_n_result<Out> format_to_n(Out out, iter_difference_t<Out> n,
                                      const locale& loc, string_viewformat-
                                      const Args&... args);
template<class Out, class... Args>
  format_to_n_result<Out> format_to_n(Out out, iter_difference_t<Out> n,
                                      const locale& loc, wstring_viewwforma
                                      const Args&... args);
```

Let

— `charT` be `decltype(fmt.str)::value_type`

...

```
template<class... Args>
  size_t formatted_size(string_viewformat-string<Args...> fmt, const Args&.
template<class... Args>
  size_t formatted_size(wstring_viewwformat-string<Args...> fmt, const Args
template<class... Args>
  size_t formatted_size(const locale& loc, string_viewformat-string<Args...
                        const Args&... args);
template<class... Args>
  size_t formatted_size(const locale& loc, wstring_viewwformat-string<Args.
                        const Args&... args);
```

Let `charT` be `decltype(fmt.str)::value_type`.

...

§ 8. Acknowledgements

Thanks to Hana Dusíková for demonstrating that the optimal formatting API can be implemented with P1221.

§ References

§ Informative References

[CODESEARCH]
Andrew Tomazos. Code search engine website. URL: https://codesearch.isocpp.org

[D-FORMAT]
D Library Reference, std.format. URL: https://dlang.org/phobos/std_format.html

[FMT]
Victor Zverovich; et al. The {fmt} library. URL: https://github.com/fmtlib/fmt

[N4861]
Richard Smith; et al. Working Draft, Standard for Programming Language C++. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/n4861.pdf

[P0645]
Victor Zverovich. Text Formatting. URL: https://wg21.link/p0645

[P1045]

David Stone. constexpr Function Parameters. URL: http://wg21.link/p1045

[P1221]

Jason Rice. Parametric Expressions. URL: http://wg21.link/p1221

[RUST-FMT]

The Rust Standard Library, Module std::fmt. URL: https://doc.rust-lang.org/std/fmt/